

# Aprendizado por Reforço através do Método Q-Learning

Luís R. Marques Júnior<sup>1</sup>  
IMECC - Unicamp, Campinas, SP

**Resumo.** Neste projeto o objetivo era treinar um agente virtual por aprendizado por reforço, com o auxílio do método Q-Learning e uma rede neural para jogar o clássico jogo da cobrinha e obter pontuações cada vez mais altas com o agente virtual.

**Palavras-chave.** Aprendizado de Máquinas, Aprendizado por Reforço, Q-Learning, Rede Neural

## 1 Introdução

Este trabalho tem como intenção a implementação computacional de uma rede neural que é treinada por meio de aprendizado por reforço, isto é, um sistema de aprendizado de máquina. No aprendizado por reforço temos um agente - nosso sistema que aprende - que realiza ações num determinado ambiente e é recompensado ou penalizado por essas ações; o aprendizado surge da melhora nas tomadas de decisão do agente.

De uma forma um pouco mais teórica, o aprendizado por reforço funciona dado um conjunto de estados  $S$ , e um conjunto de ações  $A(x)$  para cada estado  $S$ ; executando uma ação  $x$  qualquer, o agente muda de um estado para outro. Uma ação específica por estado recompensa o agente, normalmente uma pontuação. A partir disso, o objetivo é o que agente maximize essa pontuação tomando as ações  $x$  corretas em cada estado  $S$ .

---

<sup>1</sup>jr180396@gmail.com

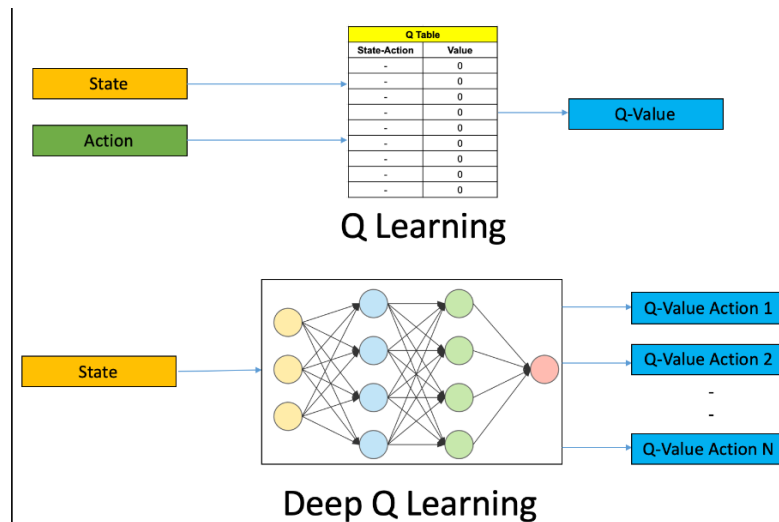


Figura 1: Modelo da Rede Neural Fonte: Google

Para o funcionamento desse sistema é necessário o auxílio de três componentes bem distintos: o agente, o modelo e o ambiente. Explicando de forma sucinta, o agente será responsável pelas ações e, consequentemente, pelo aprendizado envolvido nas suas decisões. O ambiente será o local no qual o agente atua e, por fim, o modelo será a ferramenta que irá dosar as decisões do agente, o recompensando ou penalizando por suas ações.

The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named "Projeto MS571" with files like "agente.py", "metodo\_Q.py", "cobrinha.py", and "Al.py". The code editor shows the content of "agente.py", which includes a class definition for an agent. The code includes methods for game logic, training, and decision-making. Key parts of the code include:
 

- `game.comida.y > game.head.y` for checking if the snake has eaten.
- `return np.array(estado, dtype=int)` for returning the state.
- `def guardar(self, estado,acao, recompensa, proximo_estado, done):` for saving the state, action, reward, next state, and done flag.
- `def treino longo(self):` for the long-term training loop, which includes a mini-sample and a random action.
- `def treino curto(self, estado, acao, recompensa, proximo_estado, done):` for the short-term training loop.
- `def jogada(self, estado):` for the decision-making process, which includes a random action or a predicted action based on the model.
- `def treino():` for the main training loop, which includes a total score and a done flag.

Figura 2: Trecho do código dos algoritmos Fonte: Autoria própria.

Neste trabalho o modelo escolhido foi o método Q-Learning por uma questão arbitrária de ser o modelo citado no livro-texto de referência da matéria, além de uma questão de comodidade: o ambiente escolhido foi o jogo da Cobrinha que se comporta muito bem com esse modelo.

O modelo Q-Learning foi introduzido por Chris Watkins em 1989; sendo um modelo livre - em poucas palavras, um modelo de tentativa e erro - ele não necessita de uma modelagem estrita do ambiente que será aplicado, podendo resolver problemas sem necessitar de uma adaptação pré-condicionada.

A técnica de aprendizado por reforço visa encontrar a política ótima de ações em um ambiente, assim possibilitando que o agente aprenda a tomar decisões de forma sequencial de tal modo a maximizar a recompensa ao longo do tempo. Algoritmicamente, após algumas iterações aleatórias, o Q-Learning irá tomar uma nova ação baseada no valor da função, que será equilibrada baseada na taxa de aprendizado, no valor de desconto, na recompensa adquirida e na máxima recompensa que seria possível.

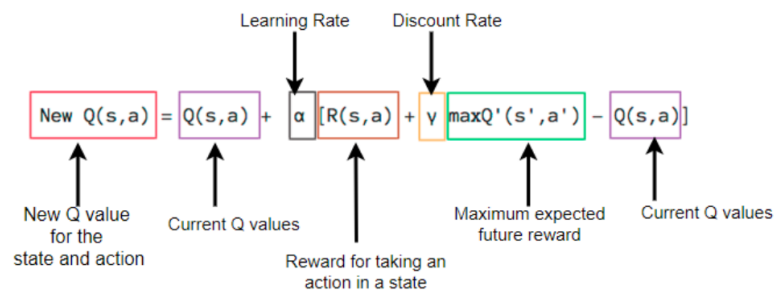


Figura 3: Equação de Bellman - Q-Learning Fonte: Google

O ambiente escolhido para o projeto foi o jogo da cobrinha; devido a sua suposta simplicidade, é um ambiente propício para o escopo desse trabalho. Bem controlado e com baixa complexidade, ideal para a demonstração de uma rede neural em funcionamento e seu aprendizado é visualmente fácil de ser demonstrado com poucas iterações.

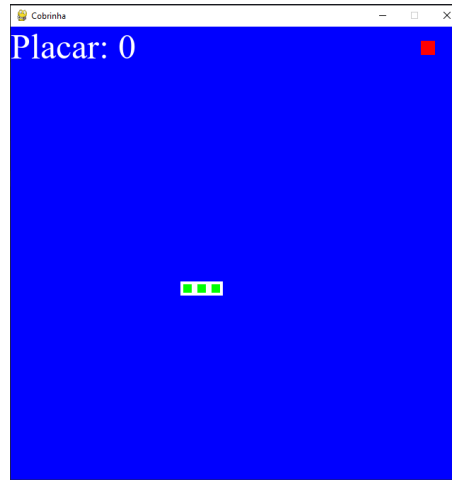


Figura 4: Ambiente - jogo da cobrinha Fonte: Autoria própria.

Por fim, o agente nada mais é que o algoritmo responsável pela interação entre nosso ambiente - jogo da cobrinha - e o modelo - Q-Learning. É de responsabilidade do algoritmo do agente a transição dos valores de cada simulação do jogo da cobrinha para o Q-Learning, que por sua vez, deve memorizar e pesar cada ação para uma melhora ao longo das iterações do agente.

## 2 Conclusão

Com a implementação dos três módulos - agente, modelo e ambiente - foi possível obter resultados bem coerentes e, acima de tudo, funcionais. O jogo da cobrinha foi corretamente implementado, porém, apesar de ser tido como a parte mais simples originalmente, pelo contrário, foi uma das mais complicadas. Exceto pelo agente, o jogo da cobrinha foi a parte mais difícil do trabalho, com relação a programação em si.

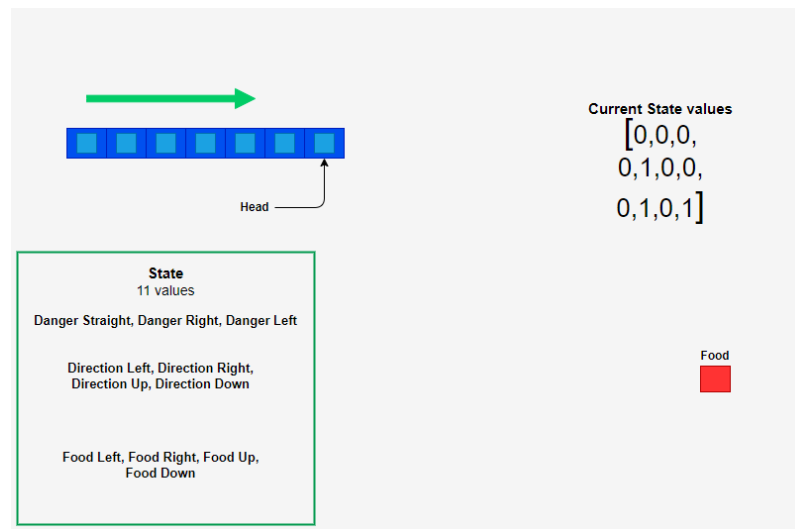


Figura 5: Estados possíveis para a posição da cobrinha Fonte: Google

Por outro lado, o Q-Learning foi bastante direto, pelo menos até sua implementação quando foi chamado pelo agente. Por fim, o agente foi a parte mais complicada. Relacionar todas as partes, fazer as chamadas das funções, detalhes de como treinar a rede neural e aplicar corretamente os pesos para a função Q-Learning foi um desafio que, na entrega desse trabalho, apesar dos bons resultados, não ficou ideal.

Tendo tudo isso em mente, o agente começou a mostrar bons resultados no final da primeira centena de iterações, e estabilizando seus resultados ao final da segunda centena. Talvez com um melhor ajuste dos hiperparâmetros ou uma mudança da dimensão do ambiente esses valores possam ser melhorados.

Claro, o resultado ainda está bem longe da aplicação patenteada em 2014, pela Google DeepMind, que usa Q-Learning em conjunto com aprendizado profundo que executa jogos do Atari 2600 em níveis sobrehumanos.

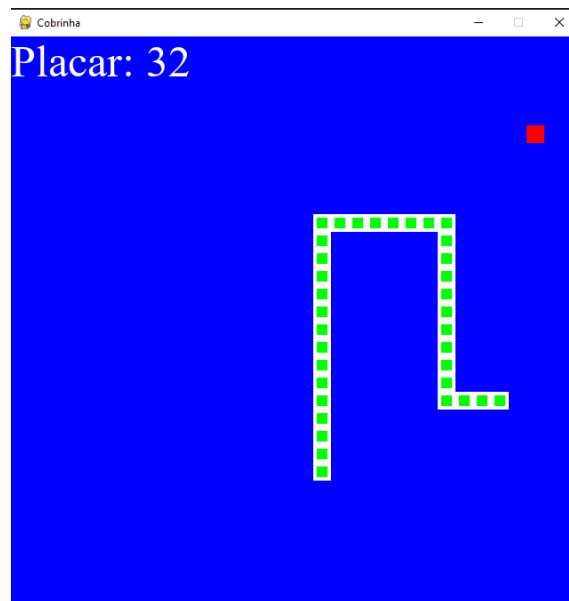


Figura 6: Resultado interessante, mas longe de ótimo. Fonte: Autoria própria.

Os códigos estão anexados junto com o .PDF do trabalho no Classroom.